

Terraform Safety Gates & Incident Automation – Usage Guide

This document explains how to safely use the provided Python and shell scripts to validate your AWS infrastructure after Terraform deployment. These scripts act as guard rails, producing PASS/FAIL evidence before higher-level automation runs.

Required Identifiers

- 1 **Instance ID** – The EC2 instance that should access the database securely (example: i-0abc123def456).
- 2 **DB Identifier** – The RDS DB instance identifier created by Terraform (example: ceklab1-db).
- 3 **Secret Name** – The AWS Secrets Manager secret holding database credentials (example: ceklab1/db).

Script Overview & Order of Execution

- 1 **gate_secrets_and_role.sh**: Verifies IAM role attachment, Secrets Manager access, and safe secret configuration.
- 2 **gate_network_db.sh**: Confirms RDS is private and only reachable via security-group-to-security-group rules.
- 3 **run_all_gates.sh**: Executes both gates and produces a combined PASS/FAIL result.
- 4 **handler.py**: Lambda function that generates incident reports using logs, SSM, Secrets, and Bedrock.
- 5 **claudex.py**: Standalone helper for manual Bedrock (Claude) queries.

How to Run the Gates

Set environment variables before running the gate scripts:

- 1 `REGION=us-east-1`
- 2 `INSTANCE_ID=i-xxxxxxxxxxxxxxxxxx`
- 3 `DB_ID=your-db-identifier`
- 4 `SECRET_ID=your/secret/name`

Then execute the combined gate runner from the script directory:

```
./run_all_gates.sh
```

Expected Results

- 1 **GREEN**: All checks passed – infrastructure is compliant.

- 2 YELLOW: Passed with warnings – review JSON outputs.
- 3 RED: Failed – Terraform configuration must be fixed.

Gate results are saved as JSON files and can be attached directly to audit or lab proof submissions.